spotifyaudiopca

October 16, 2026

[114]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

#Link to dataset
kaggle_link = "https://www.kaggle.com/datasets/maharshipandya/
 ↪-spotify-tracks-dataset?resource=download"



#loading data
df = pd.read_csv('spotifyaudio.csv')

# See all the genre names so you can pick yours
# print(df['track_genre'].unique())

#cleaning
genres = ['classical', 'pop', 'metal', 'ambient', 'country']
df = df[df['track_genre'].isin(genres)]

df = df.drop_duplicates(subset='track_id')

features = ['danceability', 'energy', 'loudness', 'speechiness',
            'acousticness', 'instrumentalness', 'liveness', 'valence', 'tempo']
df_clean = df[features + ['track_genre']]

df_clean = df_clean.dropna()

df_clean.shape #Including the genre column
```

[114]:
```
(4861, 10)
```

[115]:
```
#PART A
# The dataset I chose to use is from kaggle and is the Spotify Tracks Dataset.
# It contains a total of 21 columns consisting of information such as
# track_id, artists, album_name, track_name, popularity, duration_ms, and more.
# The main columns I will be using as features for this project are
```

```
# danceability, energy, loudness, speechiness, acousticness, instrumentalness,␣
 ↪liveness,
# valence, and tempo. The dimensions of the data matrix are (114000, 21) as of␣
 ↪now. Each row
# represents a track and each column represents information about that track in␣
 ↪terms of genre,
# artist, audio, etc. After cleaning, the dimensions of the new cleaned data␣
 ↪matrix are
# (4861, 10), which is including the genre column. The one I will
# be using will not include the genre column, making it (4861, 9). In order to␣
 ↪clean
# the original dataset, I first picked out genres that are the most different␣
 ↪from each other
# and then I extracted those genres. After that, I extracted the data that was
# related to the 9 features that I wanted, and created a new table with the␣
 ↪information
# that I need. I ended up with the following information:
print(df_clean.shape)
print(df_clean['track_genre'].value_counts())
```

```
(4861, 10)
track_genre
country      1000
ambient       999
pop           991
metal         986
classical     885
Name: count, dtype: int64
```

[116]:
```
#PART B - Calculating the Covariance Matrix

Z = df_clean.drop(columns = 'track_genre')
n = Z.shape[0]
means = Z.mean(axis = 0)
std = Z.std(axis = 0, ddof = 1)
Zc = (Z - means) / std
matrix = (Zc.T @ Zc)/(n - 1)
matrix
```

[116]:

	danceability	energy	loudness	speechiness	acousticness	\
danceability	1.000000	0.417219	0.488199	0.197245	-0.398179	
energy	0.417219	1.000000	0.854095	0.246674	-0.880417	
loudness	0.488199	0.854095	1.000000	0.158813	-0.764272	
speechiness	0.197245	0.246674	0.158813	1.000000	-0.184829	
acousticness	-0.398179	-0.880417	-0.764272	-0.184829	1.000000	
instrumentalness	-0.480761	-0.655566	-0.774552	-0.146555	0.619019	
liveness	0.030792	0.197566	0.160427	0.116087	-0.153796	

```
valence               0.571438  0.449650  0.404897     0.167209     -0.339434
tempo                 0.080543  0.279814  0.239539     0.136486     -0.260741

                  instrumentalness  liveness   valence     tempo
danceability             -0.480761  0.030792  0.571438  0.080543
energy                   -0.655566  0.197566  0.449650  0.279814
loudness                 -0.774552  0.160427  0.404897  0.239539
speechiness              -0.146555  0.116087  0.167209  0.136486
acousticness              0.619019 -0.153796 -0.339434 -0.260741
instrumentalness          1.000000 -0.142868 -0.416501 -0.210248
liveness                 -0.142868  1.000000  0.135438  0.069569
valence                  -0.416501  0.135438  1.000000  0.197147
tempo                    -0.210248  0.069569  0.197147  1.000000
```

[117]:
```
#PART C - PCs
values, vectors = np.linalg.eigh(matrix)

#sorting the eigenvalues and eigenvectors
sorted_values = values[::-1]
sorted_vectors = vectors[:, ::-1]

variance_expl = []
for i in sorted_values:
    variance_expl.append(i/sum(sorted_values))

#First five PCs seem to explain a larger portion of the variance in the data
# in comparison to the last four, with PC1 explaining 45.6% of the variance in␣
 ↪the data.

loadings = pd.DataFrame(data = sorted_vectors[:,:3],
                        index = Z.columns,
                        columns = ["PC1", "PC2", "PC3"])

print(sorted_values[:3,])
print("spread =", variance_expl[0:3])
loadings
```

```
[4.10661961 1.04986877 1.01104967]
spread = [np.float64(0.45629106815842724), np.float64(0.11665208604199453),
np.float64(0.11233885224566495)]
```

[117]:

	PC1	PC2	PC3
danceability	-0.317967	0.417900	0.423904
energy	-0.448624	-0.079694	-0.204190
loudness	-0.444449	0.047054	-0.239869
speechiness	-0.151069	-0.385856	0.615231
acousticness	0.420481	0.059818	0.288993
instrumentalness	0.403748	-0.110983	0.142308

```
liveness          -0.116891 -0.643085  0.118762
valence           -0.305701  0.231228  0.471467
tempo             -0.174192 -0.430105 -0.049102
```

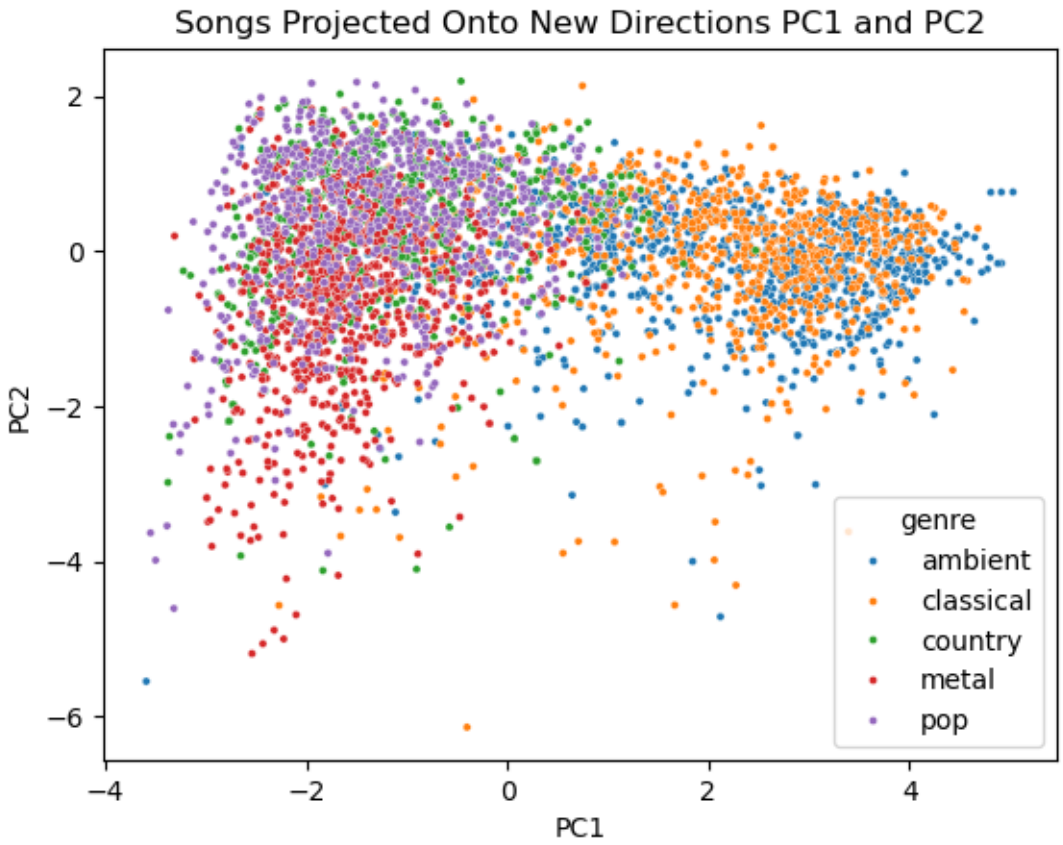
[119]:
```
#PART D
# The results from the PCA itself show us the directions
# in which the songs differ from each other. After dimension reduction, we get␣
 ↪principal
# components, which tell us how similar the songs are to one another.

#Scores tell us the new directions along each PC for each song in the original␣
 ↪dataset
scores = Zc.to_numpy() @ sorted_vectors[:, :2]

genre_labels = df_clean['track_genre'].to_numpy()

plot = pd.DataFrame(scores, columns = ['PC1', 'PC2'])
plot['genre'] = genre_labels

sns.scatterplot(data = plot, x = "PC1", y = "PC2", hue = "genre", s = 9)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("Songs Projected Onto New Directions PC1 and PC2")
plt.show()
```



PART E - FINDINGS SUMMARY

I found that about 57% of the variability was kept once PCA (dimension reduction) was performed on the dataset among the 9 features (for reduction to 2 dimensions). Overall, with 5 PCs, we are able to capture about 88% of the variability in the dataset.

According to the plot, we can see that songs have been distributed consistently with the loadings for each PC. Loadings for PC1 explain that a majority of the songs pointing in that direction are more instrumental and acoustic on the positive side and more energetic and loud on the negative side, whereas songs pointing in PC2's direction are more heavy on danceability and valence on the positive side and speechiness, liveness, and tempo on the negative side. The plot shows exactly that, with pop and metal on the lower end of PC1. We can see that pop is higher on PC2 axis, indicating pop music has a higher danceability and valence. Metal on the other hand, is lower on PC2, indicating higher speechiness, liveness, and tempo. On the PC1 axis, we can see that classical and ambient are on the higher end, indicating more acousticness and instrumentalness. Country seems to overlap with pop indicating they have similar audio features. Even though PCA did not see any of the genre labels, it was able to approximately distinguish between songs based on the audio features.